DEPLOYGUARD

Internal Engineering Handbook

Architecture, deployment contracts, runtime verification, failure recovery, operations, and certification boundaries

Document authority
This handbook is grounded in repository Hassan-Sajjad72/Deploy-Guard-dev, branch test1, source snapshot 1acd056f29eaabc607d68e6875b27c877c1bd9c9. The reusable workflow executable remains pinned to 23292cc4e0cedaf35c5b62f5f3dfbd8b0c6db37b.

Field	Value
Classification	Internal - Engineering
Document date	7 September 2026
Source branch	test1
Source snapshot	1acd056f29eaabc607d68e6875b27c877c1bd9c9
Reusable workflow pin	23292cc4e0cedaf35c5b62f5f3dfbd8b0c6db37b
Audience	Developers, DevOps/SRE, platform maintainers, reviewers, certification team
Status	Source-grounded handbook; cloud-runtime verification status is identified separately from source/local verification

Document map and reading model

This handbook deliberately combines architecture explanation, technical reference, and operational runbooks rather than treating documentation as a single narrative. The structure follows the parts of architecture documentation that matter to DeployGuard: constraints and invariants, building blocks, runtime and deployment views, cross-cutting contracts, quality/certification evidence, risks, and operational procedures.

Part	Use it when…	Primary chapters
System model	You need to understand what DeployGuard is and where authority lives.	1-5
Execution model	You need to understand how a deployment is admitted, built, verified, and promoted.	6-10
Failure & recovery	You need to classify a failure or choose Retry vs Deploy Fixed Commit.	11-14
Product surfaces	You need to know what each UI page is allowed to present.	15-16
Operations	You need a repeatable team procedure.	17-19
Maintenance & appendices	You are changing platform contracts or onboarding a new maintainer.	20 + appendices

Table of contents

1. Purpose, scope and engineering invariants

2. Source snapshot and technology baseline

3. System context and architecture

4. Canonical authorities and release truth

5. Project, service and configuration model

6. Exact-source admission, BuildTarget and port contracts

7. GitHub Actions / Railpack deployment pipeline

8. Pre-Terraform deployability validation

9. AWS runtime, verification and terminal reconciliation

10. Managed database contract

11. Lifecycle state machine

12. Canonical failure contract and evidence ownership

13. Recovery semantics

14. Historical audit vs current recovery projection

15. UI surface authority matrix

16. Observability and troubleshooting evidence

17. Security and secret boundaries

18. Operational runbooks

19. Certification and verification status

20. Change-management rules

21. Certification incident case study

22. Known boundaries and risks

Appendix A. Key source map

Appendix B. Failure-contract reference

Appendix C. Glossary

Appendix D. Documentation-method references

Research-informed documentation design

The handbook emphasizes correctness, structuredness, task-focused runbooks, and explicit source authority because maintenance and onboarding research repeatedly identifies missing, vague, outdated, or hard-to-navigate documentation as costly. The bibliography records the architecture-documentation and maintenance sources used to shape the form of this handbook.

1. Purpose, scope and engineering invariants

DeployGuard is a deployment platform whose control plane converts a user-selected GitHub repository source into a bounded, evidence-backed deployment lifecycle. Its engineering responsibility is deployability and runtime identity: build the selected source, enforce deployment contracts, provision supported infrastructure, verify the resulting runtime, and expose one coherent current state to product surfaces.

1.1 Permanent invariants

- Smallest complete change: update every canonical layer required by an invariant, while avoiding unrelated refactors.
- Architecture preservation: extend current services, entities, contracts, workflows, Terraform, and tests instead of adding competing authorities.
- Railpack authority: Railpack determines how repository application code is built and started; DeployGuard validates and carries that result rather than replacing it.
- Correctness boundary: DeployGuard proves deployability and runtime identity. Business endpoints, login behavior, domain queries, and feature correctness remain repository concerns unless they prevent deployment.
- Managed database support: admit PostgreSQL, MySQL, and MongoDB only; reject unsupported managed capabilities before infrastructure mutation.
- Generic validation: reason from observable build, process, port, binding, and database-connection behavior rather than framework/ORM-specific fixes.
- Evidence ownership: classify REPOSITORY_APPLICATION, DEPLOYGUARD_PLATFORM, EXTERNAL_PROVIDER, or UNVERIFIED from collected evidence; a stage name alone is never proof.
- Lifecycle integrity: preserve Deploy -> Redeploy -> Rollback -> Destroy state, immutable identity, and cleanup semantics.

Source anchors: .agents/skills/deployguard-p0/references/invariants.md

1.2 P0 certification envelope

Phase	P0 scope	What must be true
Control plane	P0-1 to P0-3	Selected source SHA stays independent of workflow registration; capability admission precedes Terraform; ENV scope and ownership are preserved.
Pre-Terraform	P0-4 to P0-6	Validate the exact immutable image and supported DB deployability before AWS; readiness proves deployment, not business correctness.
AWS runtime	P0-7 to P0-9	Verify AWS-only facts in AWS; collect bounded sanitized evidence; surface actionable product diagnostics.
Release truth	P0-10 to P0-11	Isolate per-service outcomes; record LIVE only after terminal reconciliation proves intended identity matches observed runtime.

Source anchors: .agents/skills/deployguard-p0/references/phase-1-control-plane.md; .agents/skills/deployguard-p0/references/phase-2-preterraform.md; .agents/skills/deployguard-p0/references/phase-3-aws-runtime.md; .agents/skills/deployguard-p0/references/phase-4-release-truth.md

2. Source snapshot and technology baseline

This edition is tied to Hassan-Sajjad72/Deploy-Guard-dev@1acd056f29eaabc607d68e6875b27c877c1bd9c9 on branch test1. That snapshot contains the global structured-failure contract repair and is the reference for all page names, lifecycle behavior, and recovery semantics described here.

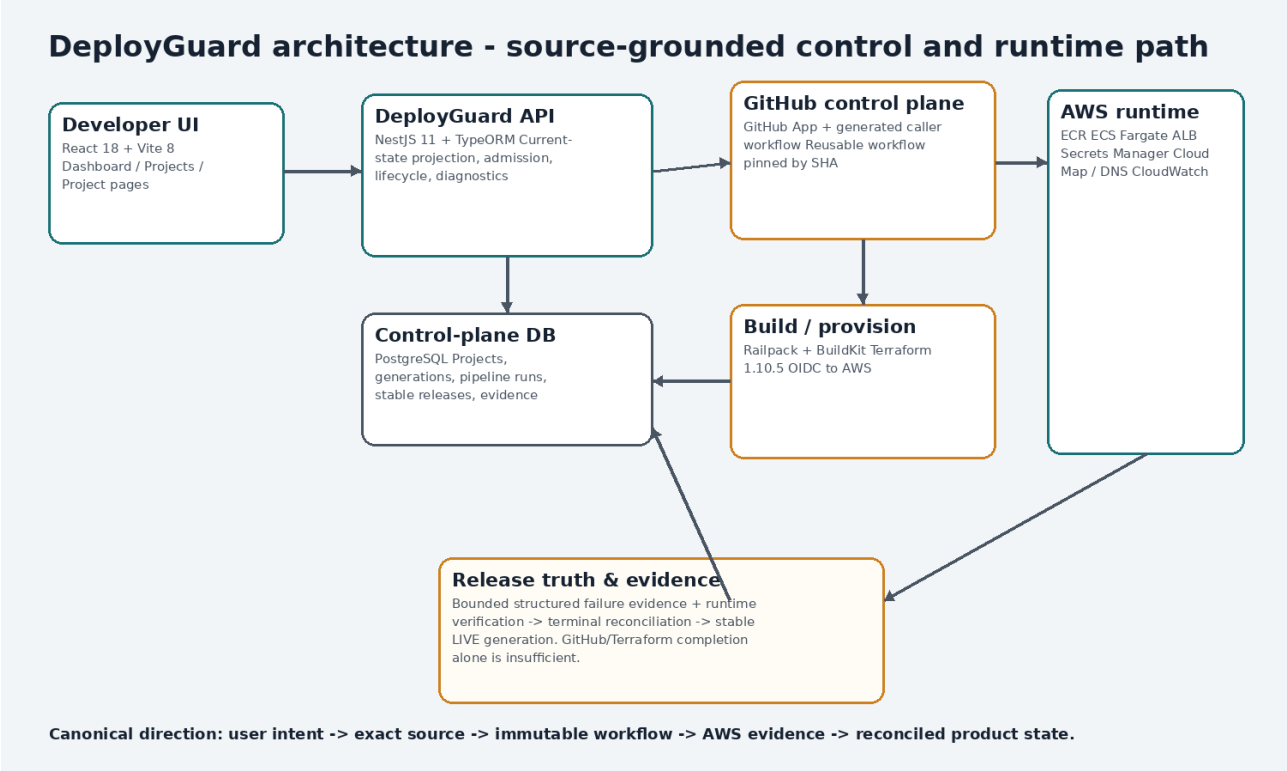
Layer	Implementation at source snapshot	Operational significance
Frontend	React 18.3.1, React Router 7.18.2, Vite 8.2.1	Authenticated product surfaces and lifecycle actions are client projections of backend current-state data.
Backend	NestJS 11, TypeORM 0.3, Node >=22 <23	Admission, project/service configuration, lifecycle dispatch/reconciliation, canonical state, diagnostics, observability.
Control-plane DB	PostgreSQL	Persists users, projects, services, runtime config revisions, generations, pipeline runs/events, stable releases, observability and audit artifacts.
Build	Railpack 0.38.0 + BuildKit container	Produces immutable service images from canonical BuildTarget and exact source.
Workflow	Reusable GitHub Actions workflow, result contract deployguard.release-result/v5	Executes immutable release operations and publishes structured terminal evidence.
Infrastructure	Terraform 1.10.5	Bootstrap/topology/destroy authority for the Railpack runtime.
AWS	ECR, ECS Fargate, ALB, Secrets Manager, Cloud Map/DNS, CloudWatch-related evidence	Runtime target and provider evidence boundary.
Auth / security	GitHub App, OIDC to AWS, Helmet, validated DTOs, role guards	Provider access and product authorization boundaries.

Source anchors: backend/package.json; frontend/package.json; backend/src/app.module.ts; backend/src/main.ts; .github/workflows/deployguard-reusable.yml

Release identity rule

The current product source HEAD and the reusable workflow executable are different identities by design. The executable remains pinned to 23292cc4e0cedaf35c5b62f5f3dfbd8b0c6db37b; documentation must not mechanically replace that pin with the current HEAD unless executable bytes/contracts require a new workflow release.

3. System context and architecture



3.1 Control-plane responsibilities

- Authenticate users and attach repository access through the configured GitHub App.
- Create one project configuration for a repository/branch/environment identity, with one or more deployable services and one canonical application entrypoint service.
- Resolve immutable source and BuildTarget/runtime configuration before dispatch.
- Persist pipeline operations and reconcile provider execution into one developer-facing current-state contract.
- Own lifecycle admission and product recovery actions; provider consoles are evidence sources, not product state authorities.
- Preserve historical evidence while projecting current recovery decisions from the current deterministic contract.

3.2 Boundary with the application repository

DeployGuard owns the deployment mechanism and deployment contracts; the repository owns application code and business behavior. A repository may be rejected when it cannot satisfy the runtime contract (for example, process exits immediately or binds only to loopback), but a successfully deployed application is not failed merely because a business endpoint returns an application error after transport and runtime identity are proven.

Source anchors: backend/src/app.module.ts; .agents/skills/deployguard-p0/references/invariants.md

4. Canonical authorities and release truth

The platform is designed around explicit authorities rather than client reconstruction. The same fact must have one source of truth and all product surfaces must consume that projection.

Fact	Canonical authority	What is explicitly not authority
Application build/start	Railpack result + canonical BuildTarget	Framework-specific guesses or client-side stack inference
Selected source	Exact 40-character source SHA resolved from selected branch/reference	Moving branch head after admission
Current developer state	ProjectCurrentStateService / stateAuthority	Frontend inference from URL presence, history rows, or isolated provider state
LIVE release	Stable release + live generation route + terminal runtime reconciliation	Terraform apply success or GitHub Actions completion alone
Application URL	applicationEntryPointServiceId projected through canonical runtime identity	Service name/order heuristics
Failure recovery	Current deterministic failure contract + persisted authoritative terminal evidence	Stale historical diagnostic snapshot by itself
Historical audit	Persisted operation/evidence/diagnostic snapshot	Current classifier overwriting old evidence
Infrastructure mutation	Terraform bootstrap/topology/destroy contract; release-only ECS activation where explicitly supported	Ad hoc UI-side cloud mutations

Source anchors: backend/src/projects/current-state/project-current-state.service.ts; backend/src/projects/current-state/project-current-state.types.ts; .agents/skills/deployguard-p0/references/phase-4-release-truth.md

5. Project, service and configuration model

5.1 Project identity and uniqueness

Project creation inspects the selected GitHub repository through the GitHub App, resolves the target branch, and prevents an active duplicate for the same owner + repository identity + target branch + environment name. Archived projects are excluded from the active-project uniqueness check. A project contains ordered deployable services and a required applicationEntryPointServiceId used by Open Application and canonical monitoring defaults.

Source anchors: backend/src/projects/projects.service.ts; backend/src/projects/projects.controller.ts

5.2 Deployable services

Property	Meaning
serviceId	Stable UUID used across runtime configuration, generations, evidence, and service-scoped failure markers.
name	User-facing service name; uniqueness and bounded input are validated.
serviceDirectory	Repository-relative path; absolute and parent-escape paths are rejected.
BuildTarget revision	Canonical resolver output binding workspace/build/install roots and execution contract to the exact source.
servicePort	Canonical application port in range 1..65535.
runtimeConfigRevisionId	Immutable runtime configuration identity for release evidence.
applicationEntryPointServiceId	The sole default public/Open Application service authority.

5.3 Environment and ownership

Configuration is service-aware and ownership-aware. DeployGuard reserves platform variables and managed-service aliases where the platform owns the corresponding binding. User values may have build, runtime, or both scope; frontend-public build variables are recognized separately from runtime secrets.

Category	Examples / rule	Owner
Platform runtime	PORT, HOST, NODE_ENV (JavaScript), AWS_REGION, DEPLOYGUARD_* identity keys	DeployGuard
Managed DB aliases	DB_*, DATABASE_*, engine-specific	DeployGuard-managed service binding

Category	Examples / rule	Owner
	aliases when a managed DB is attached	
User public build	VITE_*, NEXT_PUBLIC_*, REACT_APP_* in build/both scope	Repository/user
User runtime public	Non-secret application ENV not reserved by the platform	Repository/user
Runtime secret	Keys detected as secret or explicitly marked secret; delivered as secret reference	User input, protected delivery
External DB aliases	Database aliases without a DeployGuard-managed database attachment remain application configuration where allowed	Repository/user

Important

Reserved platform variables cannot be created, edited, deleted, or overridden through project ENV APIs. When a managed database is attached, its recognized aliases are platform-owned and submitted duplicates are ignored/rejected according to the ownership partition instead of overwriting generated connection data.

Source anchors: backend/src/projects/configuration-ownership.ts; .github/workflows/deployguard-reusable.yml

6. Exact-source admission, BuildTarget and port contracts

Admission freezes the source identity before execution. The workflow validates UUID identities, a 40-character source SHA, result-contract version, service count/uniqueness, bounded service paths, BuildTarget revision/fingerprint, service ports, ENV ownership, managed-database consistency, and rollback/destroy identities before infrastructure mutation.

6.1 BuildTarget

BuildTarget is the canonical source-topology contract. It records resolver version, status, workspace/build/install roots, package identity where relevant, package manager/execution commands for supported JavaScript topologies, and a fingerprint. Invalid, unresolved, ambiguous, or unsupported BuildTargets are explicit admission failures rather than opportunities for fallback guessing.

6.2 Port contract

- Every server service resolves one canonical port; 1..65535 is the accepted range.
- The runtime receives PORT equal to the resolved servicePort and HOST=0.0.0.0.
- Conflicting or invalid repository evidence is a repository/application failure that requires a corrected source commit.
- Runtime validation separately proves that the built process actually listens externally; resolving a port is not proof of binding.

Source anchors: .github/workflows/deployguard-reusable.yml; backend/scripts/verify-build-target-resolution.ts; backend/scripts/verify-service-port-resolution.ts

7. GitHub Actions / Railpack deployment pipeline

The reusable workflow is called with immutable operation, project, source, service, infrastructure and release-contract inputs. Concurrency is project-scoped and cancel-in-progress is disabled, preventing a newer dispatch from silently cancelling an active lifecycle operation.

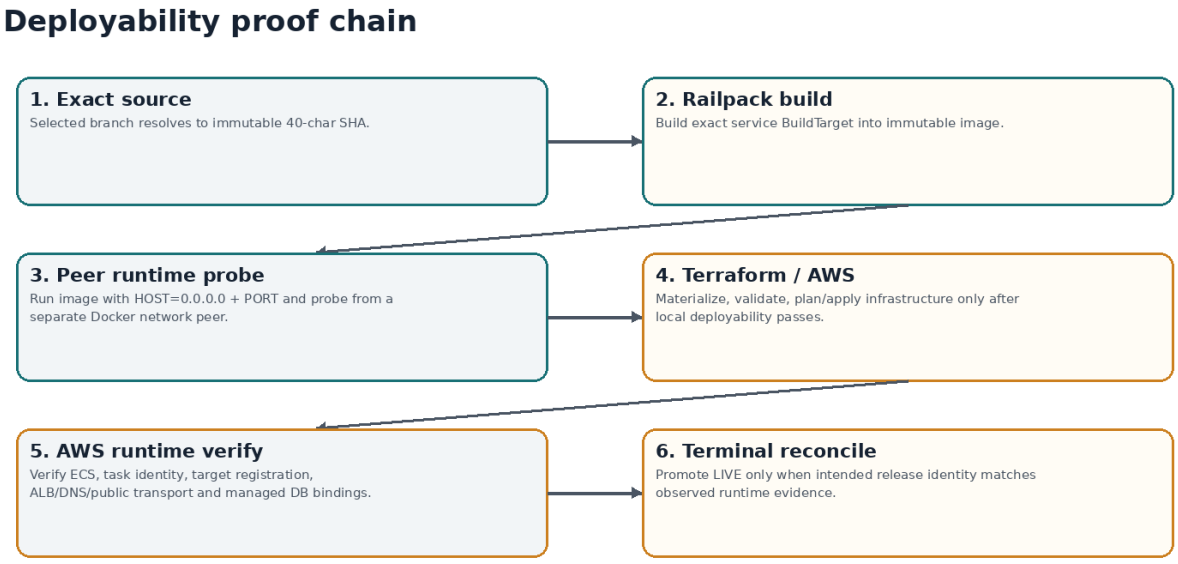
User-facing stage	Core responsibility	Mutation boundary
Checkout Source	Checkout exact application source SHA for deploy operations.	Repository read
Authenticate AWS	Acquire AWS credentials using GitHub OIDC and the configured role.	Authorization only
Validate Release	Validate immutable release input, service topology, ENV/secret ownership and contract version.	Pre-mutation
Prepare Build	Install/pin Railpack prerequisites and prepare BuildKit.	Runner-local
Build Applications	Build immutable images per service from canonical BuildTarget.	Runner-local
Validate Application Runtime	Run exact built image and prove deployability from a separate peer; validate equivalent managed DB startup contract when attached.	Runner-local + bounded provider reads for secrets
Publish Images	Create/push exact immutable images to ECR.	AWS ECR
Deploy Runtime	Materialize Terraform, validate/plan/apply or perform release-only activation according to lifecycle mode.	AWS infrastructure/runtime
Deploy Runtime and Verify Application	Verify AWS runtime identity, ECS/ALB/network/database/public endpoint evidence.	AWS reads + runtime reconciliation
Finalize Release	Publish verified result/failure evidence for control-plane reconciliation.	Evidence publication

Workflow result contract

The reusable workflow declares deployguard.release-result/v5 and emits operation-scoped artifacts. Terminal failure evidence is separately preserved with bounded size and structured DG_* markers so backend reconciliation can classify cause and ownership.

Source anchors: .github/workflows/deployguard-reusable.yml

8. Pre-Terraform deployability validation



8.1 Separate-network-peer check

The exact locally built image is started on an isolated Docker network with the service runtime environment, PORT, and HOST=0.0.0.0. DeployGuard then launches a pinned BusyBox transport-probe container on the same network and requires repeated successful TCP probes against the application network alias. This matters because probing a Docker-published host port can mask an application that only listens on 127.0.0.1 inside its container.

```
DG_FAILURE serviceId=<uuid> code=DG_APPLICATION_EXTERNAL_BINDING_FAILED
stage=application_runtime
```

8.2 Managed database probe

When a supported managed database is attached, validation starts an equivalent local engine container: PostgreSQL 16, MySQL 8, or MongoDB 8. It waits for engine readiness, materializes only the recognized aliases for that engine, and then evaluates whether the application can start under the supported connection contract. The check intentionally validates behavior rather than ORM choice.

> **Pre-Terraform policy**
> When reliable local evidence proves the immutable application cannot deploy, the deployment stops before Terraform. This is a cost and correctness boundary, not a business-feature test.

Source anchors: .github/workflows/deployguard-reusable.yml; .agents/skills/deployguard-p0/references/phase-2-preterraform.md

9. AWS runtime, verification and terminal reconciliation

Local validation cannot prove AWS-only facts. After admission succeeds, DeployGuard verifies provider-specific runtime facts and only then promotes a stable release. The AWS verification layer is intentionally distinct from pre-Terraform deployability checks.

Evidence domain	Verified fact examples
ECR	Repository/image publication and immutable digest identity.
ECS	Cluster/service identity, task definition revision, task/process existence, desired/running/pending counts.
Task network	Application is reachable on the task ENI/non-loopback path expected by downstream traffic.
ALB	ALB active, listener identity, target-group registration/health and service routing.
DNS / public transport	Public hostname resolves and endpoint accepts transport; persistent gateway reachability responses are classified separately.
Secrets Manager	Expected runtime secret identities/versions are materialized and match the admitted release.
Managed DB / Cloud Map	Database task readiness, connection materialization and service-discovery registration converge.
Release identity	Observed image/task/runtime-config/generation evidence matches intended source and release identity.

9.1 LIVE definition

Terraform apply or GitHub Actions success is not sufficient to declare LIVE. ProjectCurrentStateService projects a stable release only when the authoritative generation/release and runtime identity can be reconciled. The project state envelope distinguishes runtime state, infrastructure state, application health, monitoring availability, and reconciliation freshness.

Source anchors: .agents/skills/deployguard-p0/references/phase-3-aws-runtime.md; .agents/skills/deployguard-p0/references/phase-4-release-truth.md; backend/src/projects/current-state/project-current-state.service.ts; backend/src/projects/current-state/project-current-state.types.ts

10. Managed database contract

DeployGuard's certified managed database engines are PostgreSQL, MySQL, and MongoDB. Unsupported managed capabilities are rejected before infrastructure mutation; the platform never silently substitutes another engine.

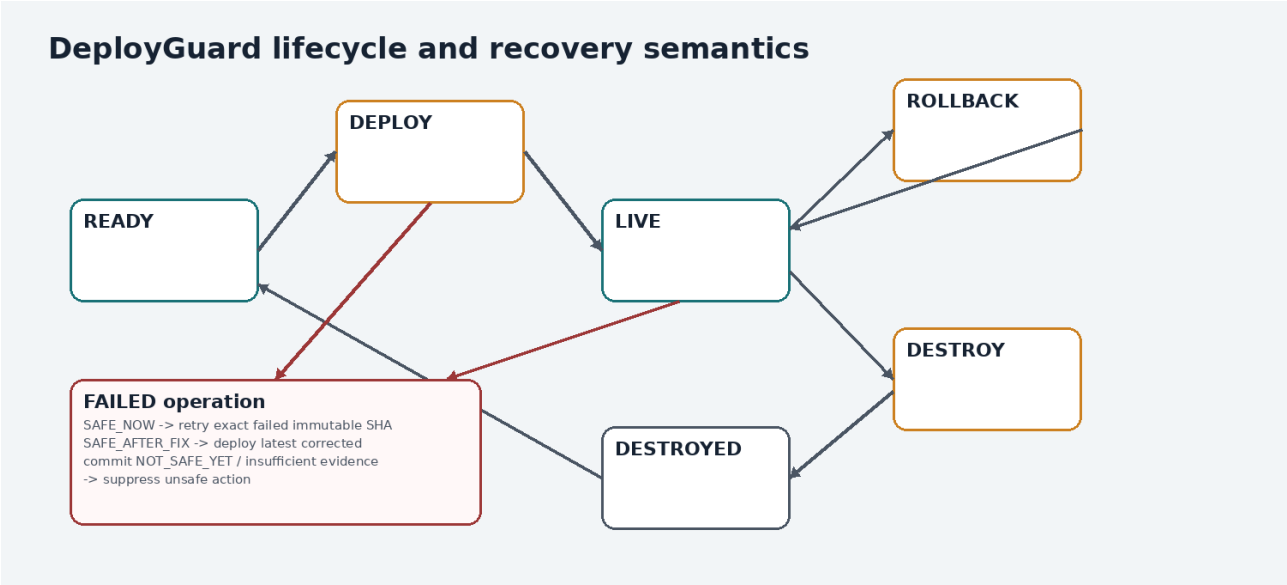
Engine	Representative aliases	Secret-bearing aliases	Validation behavior
PostgreSQL	DB_HOST, POSTGRES_HOST, PGHOST; DB_PORT/PGPORT; DB_USER/PGUSER; DB_NAME/PGDATABASE	DB_PASSWORD / PGPASSWORD; DATABASE_URL / POSTGRES_URL / POSTGRESQL_URL	Equivalent PostgreSQL 16 probe; pg_isready before app startup validation.
MySQL	DB_HOST/MYSQL_HOST; DB_PORT/MYSQL_PORT; DB_USER/MYSQL_USER; DB_NAME/MYSQL_DATABASE	DB_PASSWORD/ MYSQL_PASSWORD; DATABASE_URL/MYSQL_URL	Equivalent MySQL 8 probe; mysqladmin ping; managed grant reconciliation is separately diagnosed.
MongoDB	DB_HOST/MONGO_HOST/ MONGODB_HOST; ports/users/db aliases	MONGO/MONGODB password and URI aliases	Equivalent MongoDB 8 probe; mongosh ping under authenticated admin contract.

Single attached managed database boundary

The reusable workflow validates at most one managed-database-attached service per runtime input. Database aliases are bound only to the attached service and cannot leak across service ENV contracts.

Source anchors: backend/src/projects/configuration-ownership.ts; .github/workflows/deployguard-reusable.yml; .agents/skills/deployguard-p0/references/invariants.md

11. Lifecycle state machine



11.1 Developer states

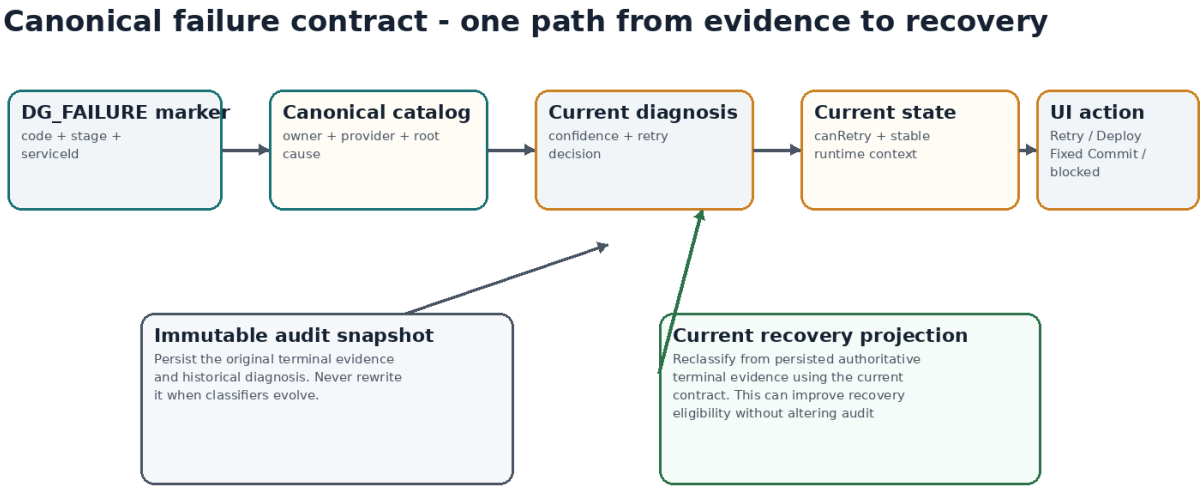
The backend developer-state model includes ready, preparing, queued, building, deploying, verifying, live, configuration_required, approval_required, failed_application, platform_attention, destroying, destroyed, and unsupported. A separate stateAuthority envelope normalizes product-facing READY / DEPLOYING / FAILED / LIVE / DESTROYING / DESTROYED / BLOCKED states and binds them to runtime/infrastructure/health/reconciliation evidence.

11.2 Lifecycle operations

Operation	Source/runtime identity rule	Expected result
Deploy	Freeze selected source SHA; build exact source.	Create candidate generation and promote when verified.
Redeploy	Normal deploy path for current/latest source when product state permits.	New release/generation or release-only activation according to canonical lifecycle.
Retry	Only for SAFE_NOW; deploy retry additionally requires original failed valid 40-char SHA.	Repeat same failed immutable operation/source, preserving retry lineage.
Rollback	Select an eligible previous stable release and its immutable runtime/source identities.	Restore a previously verified release while preserving lifecycle evidence.
Destroy	Confirmation required; remove runtime/project-owned resources and verify cleanup.	DESTROYED / project-delete-ready evidence; local cleanup may be separately retryable.
Reset Fresh	Explicit confirmation path for fresh recovery when canonical reset semantics are required.	New controlled reset/deploy operation, not a substitute for normal retry.

Source anchors: backend/src/projects/projects.controller.ts; backend/src/projects/current-state/project-current-state.types.ts; frontend/src/utils/overviewLifecyclePresentation.js

12. Canonical failure contract and evidence ownership



The current source snapshot eliminates independent failure-code mappings for known deterministic terminal codes. failure-contract.catalog.ts is the canonical catalog for codes whose marker alone proves owner/provider/diagnosis/recovery. Evidence-dependent umbrella failures such as Railpack build or ECS stability remain in FailureDiagnosticService because their underlying cause can be ambiguous.

12.1 Ownership classes

Owner	Meaning	Typical next step
REPOSITORY_APPLICATION	Evidence proves the application/repository contract is the cause.	Correct source/configuration owned by the repository; deploy a fixed commit.
DEPLOYGUARD_PLATFORM	Evidence proves a DeployGuard-owned contract/materialization/reconciliation defect or blocker.	Repair platform condition before allowing unsafe recovery.
EXTERNAL_PROVIDER	Evidence proves provider boundary such as AWS, GitHub, or Railpack.	Retry only when catalog says SAFE_NOW; otherwise resolve provider condition first.
UNVERIFIED	Evidence does not prove a more specific owner/cause.	Preserve evidence and suppress unsafe recovery actions.

12.2 Structured evidence rule

The terminal marker carries code, stage and optional serviceId. Stage is context, not proof: known deterministic codes retain their verified cause; unknown or evidence-dependent umbrella failures may remain UNVERIFIED.

Source anchors: backend/src/projects/failure-diagnostics/failure-contract.catalog.ts; backend/src/projects/failure-diagnostics/failure-diagnostic.service.ts; backend/src/projects/failure-ownership.ts

13. Recovery semantics

Retry decision	Product action	Immutable-source behavior	Use when
SAFE_NOW	Retry Failed Deployment / Rollback / Destroy	Retry uses the failed operation identity; deploy retry requires the original valid 40-char source SHA.	Cause is verified and repeating the same immutable operation is safe.
SAFE_AFTER_FIX	Deploy Fixed Commit (deploy failures only)	Uses the latest corrected repository source through normal deploy admission; does not reuse the broken SHA.	Repository/application cause requires source correction.
NOT_SAFE_YET	Action suppressed; show verified blocker	No automatic retry.	Cause is known but retry would be unsafe until a platform/provider condition is corrected.
INSUFFICIENT_EVIDENCE	Action suppressed	No automatic retry.	Cause/ownership is genuinely ambiguous or unknown.

13.1 Frontend rule

The shared failureRecoveryCommand returns retry only when currentState.canRetry and the diagnosis is SAFE_NOW. It returns deploy_fixed only for a deploy failure that is SAFE_AFTER_FIX and repository-owned. Overview and Pipeline therefore receive the same recovery meaning from the canonical current-state response.

```
SAFE_NOW + canRetry=true -> Retry Failed Deployment
SAFE_AFTER_FIX + REPOSITORY_APPLICATION + deploy -> Deploy Fixed Commit
NOT_SAFE_YET / INSUFFICIENT_EVIDENCE -> recovery command absent
```

Source anchors: frontend/src/utils/overviewLifecyclePresentation.js; backend/src/projects/current-state/project-current-state.service.ts

14. Historical audit vs current recovery projection

DeployGuard separates immutable audit history from current recovery policy. A failed operation may contain the diagnosis that was persisted when it originally failed. That snapshot is retained for audit. Current product recovery, however, is derived from authoritative terminal evidence using the current deterministic contract.

Data	Mutability / purpose
Original structured failure marker	Historical evidence; retained.
Persisted historical failureDiagnostic	Historical snapshot; retained byte-for-byte when current classifier changes.
Current diagnosis projection	Derived on read/reconciliation from current contract + persisted terminal code/stage/owner/service evidence.
canRetry	Current safety projection; can improve when a formerly unknown deterministic code becomes SAFE_NOW, subject to immutable-SHA and lifecycle gates.
UI recovery action	Current projection; must agree with current diagnosis while never rewriting historical evidence.

Why this exists

A backend restart cannot be expected to rewrite history. The correct behavior is to preserve the old diagnosis as history and compute today's recovery eligibility from immutable terminal evidence.

Source anchors: backend/src/projects/failure-diagnostics/failure-diagnostic.service.ts; backend/src/projects/current-state/project-current-state.service.ts; backend/scripts/verify-failure-diagnostics.ts; backend/scripts/verify-dispatch-state-projection.ts

15. UI surface authority matrix

Project pages are intentionally consolidated. Historical URLs redirect to current owning surfaces rather than creating competing copies of lifecycle/configuration state.

Surface / route	Primary responsibility	Must not become authority for
Dashboard /dashboard	Workspace summary, attention/live groupings, active run counts.	Reconstructing deployment state independently of current-state.
Projects /projects	Active project inventory and navigation.	Cloud cleanup or provider diagnosis.
Overview /projects/:id	Compact canonical lifecycle status and high-level actions.	Raw workflow evidence or client-derived retry safety.
Pipeline /projects/:id/pipeline	Attempt history, technical pipeline timeline, deterministic recovery evidence/actions.	Rebuilding current-state policy on the client.
Infrastructure /projects/:id/infrastructure	Canonical infrastructure evidence, Terraform/cost/runtime identity presentation.	Deployment action history.
Monitoring /projects/:id/monitoring	Runtime metrics/log/health evidence scoped to canonical application entrypoint by default.	Selecting entrypoint by name/order/UUID heuristic.
Settings /projects/:id/settings	Repository/branch/service/ENV/ database/project configuration and archive control.	Destroying AWS infrastructure by archive alone.
Troubleshooting /projects/:id/troubleshooting	Bounded evidence-assisted diagnosis for failed or LIVE-runtime contexts.	Mutating historical evidence or inventing provider ownership.
Admin /admin	Administrative/user/audit operations under admin guard.	Developer project lifecycle mutation.

15.1 Legacy route ownership

Environment/requirements bookmarks redirect to Settings; costs/state-management/storage/orchestration bookmarks redirect to Infrastructure or Pipeline as appropriate; rollback/releases/logs redirect to Pipeline; observability redirects to Monitoring. This is deliberate consolidation to keep product authority in current owning surfaces.

Source anchors: frontend/src/routes/AppRoutes.jsx; frontend/src/utils/overviewLifecyclePresentation.js

16. Observability and troubleshooting evidence

DeployGuard is designed to persist enough actionable, sanitized evidence that ordinary users can diagnose routine deployment failures inside the product. Provider consoles remain escalation evidence, not a required first stop.

Evidence class	Examples	Handling
Workflow	failed step, stage timings/status, workflow run/job identifiers	Bounded and linked to operation/attempt.
Structured failure	DG_* code, stage, serviceId, owner/provider, diagnosis, confidence, retry decision	Canonical contract-driven projection.
ECS	stopCode/stoppedReason, container exit code/reason, service events	Collect bounded relevant evidence; ECS stability ownership stays UNVERIFIED unless causal diagnostics prove more.
ALB / network	target-health reason, listener/ALB state, DNS/public transport result	Used for provider/platform classification and public-reachability diagnosis.
Database	readiness/reconciliation/secret-version state	Service-scoped and ownership-aware.
Logs	bounded CloudWatch/runtime tail where available	Sanitize/redact before persistence/display; avoid unlimited raw-log mirroring.
AI troubleshooting	Evidence-bound session snapshot with current recovery projection	Reuse evidence snapshot; classifier updates may improve current interpretation without rewriting original snapshot.

Source anchors: .agents/skills/deployguard-p0/references/phase-3-aws-runtime.md; backend/src/projects/current-state/project-current-state.types.ts

17. Security and secret boundaries

Boundary	Current design
HTTP hardening	NestJS bootstrap applies Helmet. CORS origin defaults to local Vite URL unless FRONTEND_URL overrides it.
Input validation	Global ValidationPipe uses whitelist=true, forbidNonWhitelisted=true, transform=true.
Authentication attachment	AuthenticatedUserMiddleware applies to all routes; public health/OAuth entry points continue anonymously where designed.
Role authorization	Project APIs are guarded for ADMIN / DEVELOPER / READONLY with write operations additionally restricted; /deploy requires developer role and admin surfaces use admin protection.
GitHub access	GitHub App installation/token is used to inspect repositories and exact source; selected source branch remains separate from workflow-registration requirements.
AWS authentication	Reusable workflow requests id-token: write and assumes the configured AWS role via OIDC.
Secret delivery	Secret values are materialized from exact Secrets Manager references/version identities; user project configuration cannot override reserved platform secrets.
Evidence sanitization	Runtime failure evidence is bounded and secret-like tokens are redacted before persistence/presentation.
Terraform plan for cost evidence	Sensitive values are redacted from generated plan JSON before cost-processing artifacts are exposed.

Internal-document rule

Never paste production credentials, secret values, GitHub installation tokens, AWS session credentials, decrypted ENV values, or unredacted runtime logs into this handbook. Document identifiers/contracts, not secrets.

Source anchors: backend/src/main.ts; backend/src/app.module.ts; backend/src/projects/projects.controller.ts; backend/src/projects/configuration-ownership.ts; .github/workflows/deployguard-reusable.yml

18. Operational runbooks

18.1 New deployment

1. Confirm GitHub App access to the intended repository and select the exact branch.
2. Configure deployable services, service directories, and application entrypoint service.
3. Configure service ENV/secrets and optional supported managed database attachment; resolve any admission blockers.
4. Initiate Deploy. Record the operation ID and immutable source SHA.
5. Use Pipeline for execution evidence. Treat pre-Terraform application-runtime failure as repository/application evidence when the canonical diagnosis says so.
6. Declare success only when Overview/current-state is LIVE and stable runtime identity/public URL have been reconciled.

18.2 Repository failure -> Deploy Fixed Commit

7. Confirm diagnosis owner is REPOSITORY_APPLICATION and retryDecision is SAFE_AFTER_FIX.
8. Fix the repository-owned cause and push a new commit.
9. Use Deploy Fixed Commit. This runs normal deploy admission against the corrected current source; it is not an immutable retry of the broken commit.
10. Verify the new attempt and terminal reconciliation.

18.3 Provider/transient failure -> Retry

11. Confirm current diagnosis is SAFE_NOW and the Pipeline/Overview exposes Retry.
12. For deploy retry, confirm the failed operation has a valid original 40-character source SHA.
13. Retry the failed operation; verify retry lineage points to the prior operation and source identity is unchanged.
14. If the action is absent, do not bypass the safety gate; inspect the verified blocking reason or insufficient evidence.

18.4 Rollback

15. Verify a rollback candidate exists from a previous stable release.
16. Select the target operation/release through the canonical rollback action.
17. Track Pipeline and verify restored runtime identity before treating rollback as successful.
18. A rollback failure can be retried only when its current diagnosis is SAFE_NOW.

18.5 Destroy

19. Use Destroy Infrastructure and provide the required confirmation phrase.
20. Track Terraform/runtime deletion and project-owned ECR/secret/state cleanup evidence.
21. Treat destroy as complete only when deletion verification succeeds.
22. If cloud deletion succeeded but local final cleanup fails with DG_PROJECT_DELETE_CLEANUP_FAILED, use the safe local cleanup retry path rather than redispatching destructive infrastructure deletion.

18.6 Archive

Archive is a project-record lifecycle action, not an infrastructure destroy operation. The service implementation marks status ARCHIVED and sets archivedAt. Team procedure must therefore destroy verified cloud resources first when resource cleanup is required, then archive the project record.

Do not conflate Archive and Destroy
Archiving removes a project from active workspace lists. It does not constitute evidence that AWS resources were destroyed.

18.7 Release identity check

23. Record current DeployGuard source HEAD (1acd056f29eaabc607d68e6875b27c877c1bd9c9 for this handbook edition).
24. Record current reusable workflow pin (23292cc4e0cedaf35c5b62f5f3dfbd8b0c6db37b for this edition).
25. Run the repository release-identity verification before changing a pin.
26. Change the pinned workflow SHA only when the reusable-workflow executable/contract requires a new release; preserve a byte-identical certified pin otherwise.

Source anchors: backend/src/projects/projects.controller.ts; backend/src/projects/projects.service.ts; frontend/src/utils/overviewLifecyclePresentation.js; backend/scripts/verify-release-identity.ts

19. Certification and verification status

DeployGuard uses executable verification rather than a claim of absolute defect freedom. The repository exposes a layered certification suite covering source architecture, contracts, lifecycle properties, UI presentations, release identity, Terraform/workflow contracts, and real full-stack certification hooks.

Verification layer	Representative checks from package scripts	Status for documented source
Build integrity	backend build; frontend Vite build	Reported PASS for global failure-contract repair
Architecture	dependency-cruiser + no-legacy-architecture	Part of full local certification suite
Admission/contracts	configuration admission, UUID, service port, BuildTarget, deployment requirements	Part of source/local certification suite
Lifecycle	unified Railpack lifecycle; dispatch/current-state; destroy authority; lifecycle properties	Reported PASS; latest repair report included 1,000 state runs + 250 runtime runs
Failure/recovery	failure ownership, diagnostics/global matrix, terminal observability, Pipeline/Overview/Troubleshooting recovery UI	Reported PASS for source snapshot change
Runtime identity	multi-service release/runtime, immutable revisions, terminal runtime bindings, live-runtime authority	Part of local certification suite
Release identity	external action pins + verify-release-identity	Reported PASS; workflow executable pin unchanged
Cloud runtime	real AWS deployment/redeploy/retry/rollback/destroy	Must be recorded separately per certification run; latest global failure-contract change did not itself perform AWS mutation

Certification language

Use “source-verified”, “locally verified”, “runtime verified”, or “UNVERIFIED” precisely. Never translate a local test PASS into proof of AWS-only behavior. A defensible release statement is “certified within the tested envelope”, not “100% defect-free”.

Source anchors: backend/package.json; frontend/package.json; .agents/skills/deployguard-p0/references/phase-3-aws-runtime.md

20. Change-management rules

20.1 Adding or changing a DG_* terminal failure code

27. Identify the exact authoritative emitter and define whether code alone proves owner/provider/recovery or whether the code is an evidence-dependent umbrella.
28. For deterministic codes, add/update the single canonical failure contract catalog rather than parallel mappings.
29. Ensure terminal marker retains code, stage, and serviceId where service-scoped.
30. Add regression asserting owner, provider, root cause, confidence, retry decision, recommended action and frontend recovery command.
31. Add a global audit assertion that every executable emitted code is either present in the canonical catalog or explicitly listed as evidence-dependent.
32. Preserve unknown/ambiguous evidence as UNVERIFIED; never invent ownership from a stage name.
33. Verify historical audit snapshots remain immutable while current recovery projection uses the current contract.

20.2 Changing lifecycle behavior

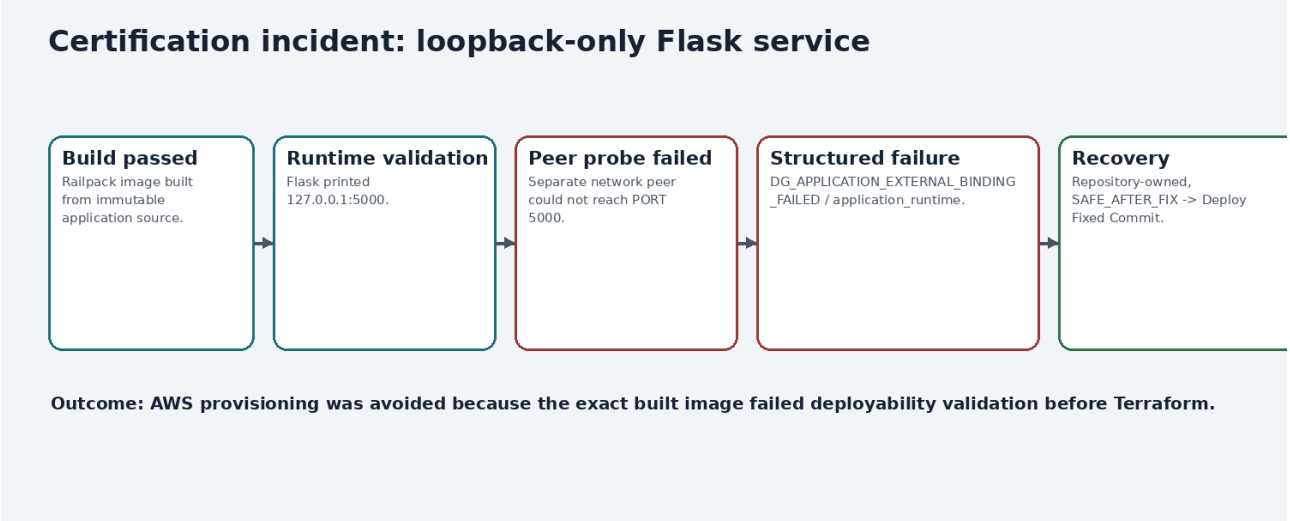
34. Update backend state authority/current-state projection first.
35. Update dispatch/reconciliation and immutable identity rules.
36. Update shared frontend presentation helper rather than one page in isolation.
37. Add backend + Overview/Pipeline/Troubleshooting regressions that prove all surfaces agree.
38. Run lifecycle property tests and destroy/rollback authority checks.

20.3 Changing the reusable workflow

39. Treat executable workflow bytes as a release artifact.
40. Run workflow executable contract and release-identity checks.
41. Publish a new immutable workflow commit only when executable change requires it.
42. Update backend/runtime pin through the existing release-identity contract, then run local certification before live AWS testing.

Source anchors: backend/src/projects/failure-diagnostics/failure-contract.catalog.ts; backend/scripts/verify-failure-diagnostics.ts; backend/scripts/verify-dispatch-state-projection.ts; frontend/scripts/verify-overview-lifecycle.mjs; frontend/scripts/verify-pipeline-recovery.mjs

21. Certification incident case study: loopback-only Flask service



A real certification deployment of Hassan-Sajjad72/log-management-analytics-system produced a decisive pre-Terraform failure. The Railpack build passed, then Flask reported it was serving on http://127.0.0.1:5000. The separate network-peer validator could not reach the application on PORT 5000 within the bounded period and emitted DG_APPLICATION_EXTERNAL_BINDING_FAILED at stage application_runtime.

```
* Running on http://127.0.0.1:5000
DG_FAILURE serviceId=... code=DG_APPLICATION_EXTERNAL_BINDING_FAILED stage=application_runtime
Application was not reachable from a separate network peer on PORT=5000 within 45 seconds.
```

21.1 What the incident proved

- The new validator detected the application-owned external-binding defect before ECR publication/Terraform provisioning.
- The correct remediation is repository-owned: bind the application to 0.0.0.0 and consume platform PORT, then deploy a corrected commit.
- The incident also exposed a product-level diagnostic consistency defect: ownership recognized the new code while the central diagnosis initially fell through to unverified.
- The global repair at source snapshot 1acd056f centralized deterministic failure contracts and added cross-layer regressions so recognized structured codes cannot diverge between ownership, diagnosis, retry policy and UI recovery.
- The final recovery semantic for this case is REPOSITORY_APPLICATION -> DETERMINISTIC -> SAFE_AFTER_FIX -> Deploy Fixed Commit.

Presentation lesson

This is a useful demonstration because the platform both stopped an invalid deployment early and preserved evidence explaining why. The later failure-contract repair is documented as a platform defect discovered and closed during certification, not hidden as an application issue.

Source anchors: .github/workflows/deployguard-reusable.yml; backend/src/projects/failure-diagnostics/failure-contract.catalog.ts; backend/scripts/verify-failure-diagnostics.ts

22. Known boundaries and risks

Boundary / risk	Team interpretation
Cloud evidence freshness	AWS observations can be unavailable/stale; current-state exposes source/freshness instead of fabricating certainty.
Evidence-dependent umbrella failures	DG_RAILPACK_BUILD_FAILED and DG_ECS_STABILITY_FAILED may remain UNVERIFIED until unique causal evidence exists.
Application correctness	A deployable runtime can still contain business defects; DeployGuard intentionally does not turn business QA into deployment admission.
Managed DB scope	Certified engines are PostgreSQL, MySQL, MongoDB. Redis/storage alias knowledge in configuration ownership is not equivalent to certified managed-service admission.
Monorepo/project uniqueness	An active duplicate for the same owner/repository/branch/environment is rejected. Test fixtures containing many independent apps in one repository should be split or modeled intentionally rather than assumed to create independent duplicate projects.
Archive vs infrastructure	Archive hides/retains project record status; it is not a cloud-destroy proof.
Workflow/source identity	Current application source, backend source, caller workflow registration, and reusable workflow executable SHA are distinct identities; mixing them is a release-integrity risk.
Documentation drift	This handbook is source-snapshot-specific. Changes to routes, contracts, failure catalog or workflow require handbook review.

Source anchors: backend/src/projects/current-state/project-current-state.types.ts; backend/src/projects/projects.service.ts; backend/src/projects/configuration-ownership.ts; .agents/skills/deployguard-p0/references/invariants.md

Appendix A. Key source map

Path	Why a maintainer reads it
backend/src/app.module.ts	Backend modules/entities and PostgreSQL control-plane registration.
backend/src/main.ts	HTTP bootstrap, Helmet, CORS, validation.
backend/src/projects/projects.controller.ts	Project/configuration/lifecycle API surface.
backend/src/projects/projects.service.ts	Project/repository/service/ENV domain rules and archive semantics.
backend/src/projects/current-state/project-current-state.service.ts	Canonical developer/product state projection and recovery eligibility.
backend/src/projects/current-state/project-current-state.types.ts	StateAuthority, developer state, infrastructure evidence and generation contracts.
backend/src/projects/configuration-ownership.ts	Reserved ENV, secret detection, managed-service aliases and ownership partition.
backend/src/projects/failure-diagnostics/failure-contract.catalog.ts	Single source for deterministic failure owner/provider/diagnosis/recovery.
backend/src/projects/failure-diagnostics/failure-diagnostic.service.ts	Evidence-dependent diagnostics and current recovery projection.
backend/src/projects/failure-ownership.ts	Structured marker extraction and evidence ownership boundary.
backend/src/projects/railpack-deployment.service.ts	Dispatch, retry, rollback, destroy, reconciliation and immutable lineage.
.github/workflows/deployguard-reusable.yml	Immutable release execution, build/runtime validation, Terraform/AWS operations, result artifacts.
infrastructure/railpack-runtime/verify-runtime.sh	AWS runtime verification and public/service evidence.
infrastructure/railpack-runtime/register-release-task-definitions.sh	Release-only ECS task-definition activation.
infrastructure/railpack-runtime/build-release-result.sh	Verified release result construction.
frontend/src/routes/AppRoutes.jsx	Current page/routes and legacy redirect ownership.
frontend/src/utils/overviewLifecyclePresentation.js	Shared canonical lifecycle copy/actions/recovery commands.
frontend/src/components/projects/PipelineExecution.jsx	Pipeline attempt execution and recovery command UI.
backend/scripts/verify-failure-diagnostics.ts	Global failure-contract matrix and historical projection regressions.
backend/scripts/verify-dispatch-state-projection.ts	Current-state/retry safety projection regressions.
backend/scripts/verify-release-identity.ts	Reusable workflow release identity contract.

Appendix B. Failure-contract reference

The deterministic catalog contains the authoritative known codes whose code alone proves owner/provider/diagnosis/recovery. The latest audit reported 42 authoritative/evidence-aware terminal codes with zero missing diagnosis, ownership mismatch, retry mismatch, or presentation mismatch. The table below is the operational subset most likely to be used during deployment certification; maintainers must consult the catalog for the complete current set.

Code	Owner/provider	Decision	Operational meaning
DG_DEPLOYMENT_INPUT_REQUIRED	DeployGuard	NOT_SAFE_YET	Required service-scoped configuration is missing before dispatch.
DG_BUILD_TARGET_UNRESOLVED / AMBIGUOUS / UNSUPPORTED / INVALID	DeployGuard	NOT_SAFE_YET	Canonical exact-source BuildTarget cannot be safely admitted.
DG_SERVICE_PORT_UNRESOLVED / CONFLICT / INVALID	Repository	SAFE_AFTER_FIX	Repository port contract requires correction.
DG_APPLICATION_RUNTIME_FAILED	Repository	SAFE_AFTER_FIX	Structured application startup/runtime failure.
DG_APPLICATION_EXTERNAL_BINDING_FAILED	Repository	SAFE_AFTER_FIX	Exact image failed separate-peer 0.0.0.0:PORT reachability.
DG_LOCAL_HOST_PORT_ALLOCATION_FAILED	DeployGuard	SAFE_NOW	Runner-local probe port allocation problem.
DG_WORKFLOW_CONTRACT_INVALID	DeployGuard	NOT_SAFE_YET	Immutable workflow input/result contract mismatch.
DG_CONTROL_PLANE_VERSION_MISMATCH	DeployGuard	NOT_SAFE_YET	Control-plane and workflow release identities incompatible.
DG_TERRAFORM_MATERIALIZATION_FAILED / VALIDATE_FAILED / PLAN_FAILED	DeployGuard	NOT_SAFE_YET	Platform-generated Terraform materialization/validation/planning defect.
DG_TERRAFORM_APPLY_FAILED	AWS provider	NOT_SAFE_YET	Apply boundary failed; provider condition must be resolved before retry.
DG_ECR_PUBLISH_FAILED	AWS provider	NOT_SAFE_YET	Image publication failed.
DG_AWS_AUTHORIZATION_FAILED	AWS provider	NOT_SAFE_YET	Required AWS authorization rejected.
DG_AWS_PROVIDER_FAILED	AWS provider	SAFE_NOW	Transient/provider operation failure is safely repeatable.
DG_RUNTIME_SECRET_MATERIALIZATION_FAILED	AWS provider	NOT_SAFE_YET	Secrets Manager runtime materialization did not complete.
DG_CLOUD_MAP_REGISTRATION_TIMEOUT	AWS provider	SAFE_NOW	Service discovery did not converge in the bounded period.
DG_ALB_NOT_ACTIVE	AWS provider	SAFE_NOW	ALB did not become active in bounded convergence.
DG_PUBLIC_DNS_UNRESOLVED	AWS provider	SAFE_NOW	Public ALB DNS did not resolve in

Code	Owner/provider	Decision	Operational meaning
			time.
DG_PUBLIC_CONNECTION_FAILED	AWS provider	SAFE_NOW	Public endpoint did not accept a transport connection in time.
DG_PUBLIC_REACHABILITY_FAILED	AWS provider	SAFE_NOW	Persistent gateway reachability result after bounded convergence.
DG_MANAGED_DATABASE_READINESS_FAILED	DeployGuard	NOT_SAFE_YET	Platform-owned managed DB readiness verification failed.
DG_MANAGED_MYSQL_GRANT_RECONCILIATION_FAILED	DeployGuard	NOT_SAFE_YET	Managed MySQL grants could not be reconciled.
DG_RELEASE_FINALIZATION_FAILED	DeployGuard	SAFE_NOW	Workflow succeeded but local stable-release projection failed.
DG_PROJECT_DELETE_CLEANUP_FAILED	DeployGuard	SAFE_NOW	Cloud deletion verified; retry local control-plane cleanup.
DG_RAILPACK_BUILD_FAILED	Evidence-dependent	INSUFFICIENT or derived	Umbrella boundary; underlying repository/provider cause must be established from unique evidence.
DG_ECS_STABILITY_FAILED	Evidence-dependent	INSUFFICIENT or derived	Umbrella boundary; ownership follows causal ECS diagnostics, not stage name.

Source anchors: backend/src/projects/failure-diagnostics/failure-contract.catalog.ts; backend/src/projects/failure-diagnostics/failure-diagnostic.service.ts

Appendix C. Glossary

Term	DeployGuard meaning
BuildTarget	Canonical exact-source build topology and execution contract for a service.
Current state	Single backend projection consumed by product surfaces; includes stateAuthority, latest attempt, stable release, recovery and evidence context.
Generation	Infrastructure/runtime generation with lifecycle status and Terraform state key.
Stable release	Verified release attached to the authoritative live generation.
Source SHA	Immutable 40-character application source commit identity.
Control-plane SHA	Immutable DeployGuard source used to materialize the workflow runtime contract.
Workflow pin	Published reusable-workflow executable SHA referenced by generated caller workflows.
RuntimeConfigRevision	Immutable service runtime configuration identity.
SAFE_NOW	Repeating the same immutable failed operation is currently safe under the contract.
SAFE_AFTER_FIX	Repository/application source must change; use normal fixed-commit deployment.
NOT_SAFE_YET	Cause is known, but automated retry is suppressed until the proven blocker changes.
INSUFFICIENT_EVIDENCE	Cause/ownership remains ambiguous; unsafe recovery is intentionally suppressed.
Terminal reconciliation	Final comparison of intended immutable release identity with observed runtime/provider evidence before LIVE.
Archive	Project-record status transition that removes the project from active lists; distinct from infrastructure Destroy.
Reset Fresh	Explicit confirmed lifecycle recovery path that resets/redeploys under its own admission semantics.

Appendix D. Documentation-method references

These references influenced the structure and maintenance philosophy of this handbook. They do not define DeployGuard behavior; repository source and executable verification remain the product authority.

Reference	How it influenced this handbook	URL / DOI
arc42	Architecture documentation template emphasizing goals, constraints, context, building blocks, runtime view, deployment view, cross-cutting concepts, decisions, quality, risks, and glossary.	https://arc42.org/overview/
arc42 Documentation	Practical guidance on avoiding documentation that is outdated, chaotic, excessive, or difficult to maintain; advocates documentation close to source.	https://arc42.org/documentation/
Diataxis	Separates documentation needs into tutorials, how-to guides, technical reference, and explanation. This handbook combines explanation, reference, and how-to/runbook content in clearly separated sections.	https://www.diataxis.fr/
Google SRE Incident Management Guide	Operational documentation should support prepared, repeatable incident response and evidence-driven mitigation.	https://sre.google/resources/practices-and-processes/incident-management-guide/
de Souza, Anquetil & de Oliveira (2006)	Survey of maintainers on which documentation artifacts matter for software maintenance and the question of how much documentation is enough. Journal of the Brazilian Computer Society 12, 31-44. DOI 10.1007/BF03194494.	https://doi.org/10.1007/BF03194494
Lethbridge, Singer & Forward (2003)	Empirical work on how software engineers use and update documentation, emphasizing real maintenance practice. IEEE Software 20(6), 35-39. DOI 10.1109/MS.2003.1241364.	https://doi.org/10.1109/MS.2003.1241364
How OSS communities document software architecture (2014)	Survey of architecture documentation in 2,000 open-source projects; found architecture documentation scarce and context-dependent. ICECCS 2014. DOI 10.1109/ICECCS.2014.26.	https://doi.org/10.1109/ICECCS.2014.26

Maintenance policy

Review this handbook whenever the canonical routes, state authority, failure catalog, workflow result contract, reusable workflow pin, supported managed databases, or lifecycle operations change. At minimum, stamp the new source snapshot and update affected source anchors.

END OF HANDBOOK

Source authority: Hassan-Sajjad72/Deploy-Guard-dev@1acd056f29eaabc607d68e6875b27c877c1bd9c9
Reusable workflow pin: 23292cc4e0cedaf35c5b62f5f3dfbd8b0c6db37b
Generated 7 September 2026